

```
// /api/create-checkout-session.js
// Vercel Serverless Function (Node.js).
Erstellt eine Stripe-Checkout-Session
// serverseitig. Der Stripe SECRET KEY wird
NIE im Frontend-Code verwendet,
// sondern nur hier über eine
Umgebungsvariable (siehe Setup-Anleitung).
//

// Preise werden HIER serverseitig
festgelegt (nicht vom Client übernommen!),
// damit niemand über die Browser-Konsole
einen anderen Preis erzwingen kann.

const Stripe = require('stripe');
const stripe =
Stripe(process.env.STRIPE_SECRET_KEY);

// Preise in Rappen (kleinste Einheit für
CHF bei Stripe)
const PRICES = {
  individual: { amount: 4900, name:
'Individual-Analyse · Astrosophie' },
  seelenkompass: { amount: 9900, name:
'SeelenKompass · Astrosophie' },
  jahreshoroskop_once: { amount: 9900,
name: 'Jahreshoroskop 2026 · Astrosophie'
},
  jahreshoroskop_monthly: { amount: 900,
name: 'Jahreshoroskop 2026 · Monatsabo ·
Astrosophie' }
```

```

};

module.exports = async (req, res) => {
  // CORS: nur nötig falls die Seite von
  // einer anderen Domain aus aufruft
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');
  if (req.method === 'OPTIONS') {
    res.status(200).end(); return; }
  if (req.method !== 'POST') {
    res.status(405).json({ error: 'Method not allowed' }); return; }

  try {
    const { pkg, payMode, returnUrl } =
    req.body || {};
    if (!pkg || !returnUrl) {
      res.status(400).json({ error: 'pkg und returnUrl sind erforderlich' }); return; }

    let priceInfo, isRecurring = false;
    if (pkg === 'individual') { priceInfo = PRICES.individual; }
    else if (pkg === 'seelenkompas') {
      priceInfo = PRICES.seelenkompas; }
    else if (pkg === 'jahreshoroskop') {
      isRecurring = (payMode === 'monthly');
      priceInfo = isRecurring ?
      PRICES.jahreshoroskop_monthly :

```

```

PRICES.jahreshoroskop_once;
    } else {
        res.status(400).json({ error:
'Unbekanntes Paket: ' + pkg });
        return;
    }

    const priceData = {
        currency: 'chf',
        unit_amount: priceInfo.amount,
        product_data: { name: priceInfo.name
    }
    };
    if (isRecurring) { priceData.recurring
= { interval: 'month' }; }

    // returnUrl kommt vom Client OHNE die
paid/session_id-Parameter -
    // die hängen wir hier serverseitig an,
damit der Client sie nicht fälschen kann.
    const successUrl = returnUrl +
(returnUrl.indexOf('?') > -1 ? '&' : '?') +
'paid=1&session_id={CHECKOUT_SESSION_ID}';
    const cancelUrl = returnUrl +
(returnUrl.indexOf('?') > -1 ? '&' : '?') +
'paid=0';

    const session = await
stripe.checkout.sessions.create({
        mode: isRecurring ? 'subscription' :
'payment',
        payment_method_types: ['card',
'twint'],
        line_items: [{ price_data: priceData,

```

```
quantity: 1 }],
    success_url: successUrl,
    cancel_url: cancelUrl,
    // Erlaubt Kündigung durch Kunden
    selbst über den Stripe Customer Portal,
    // falls Portal aktiviert ist
    (empfohlen bei Abos, siehe Setup-Hinweise).
    ...(isRecurring ? {} : {})
  });

  res.status(200).json({ url: session.url
});
} catch (err) {
  console.error('Stripe-Fehler:', err);
  res.status(500).json({ error:
err.message || 'Unbekannter Stripe-Fehler'
});
}
};
```